Preprint draft · Method and analysis

Evidence-Weighted Confidence

Auditable, Calibrated Scoring of LLM Outputs

Issam Zeinoun
Independent Researcher

October 2026

Abstract

A confidence number reported by a large language model (LLM) has no defined meaning: nothing ties "0.9" to how often the model is right. We propose *evidence-weighted confidence*, a method that replaces self-reported confidence with a score computed from verifiable evidence. A domain rubric, derived from the rules that govern the decision, breaks each output into evidence questions; for medical coding, those rules are the ICD-10-CM Official Guidelines for Coding and Reporting and the code's own instructional notes. The LLM answers each question and quotes the supporting source text; code verifies the quote and sums pre-set weights for each answer. Because the weights are log likelihood ratios, the sum is a Bayesian posterior log-odds, and fitting the weights on labeled data makes the score a calibrated probability.

The method is designed to give LLM validation four things that self-reported confidence lacks: a calibrated probability in place of an overconfident verbalized number; an audit trail of verified quotes and rubric answers behind every score; separate weights for silence and contradiction in the source; and threshold routing, in which outputs above a set probability are accepted automatically and the rest go to human review. We illustrate it with one example application, ICD-10-CM coding of E11.22 (type 2 diabetes with diabetic chronic kidney disease), and outline an evaluation protocol. Rubrics are use-case specific, but the scoring theory and the protocol are general.

Keywords: LLM evaluation; calibration; uncertainty; auditability; weight of evidence; logistic regression; medical coding; risk adjustment.

Note: this version presents the method and its analysis. Experimental results will follow in a later version.

1 Introduction

A confidence score is a claim about frequency: of all outputs scored 0.9, about 90% should be correct. A number the model writes in its answer makes no such claim, because nothing in its production was checked against outcomes. Empirical work confirms this. Xiong et al. [1] found that verbalized confidence clusters in the 80–100% range in multiples of 5 and is poorly calibrated. Later work reports that RLHF-tuned models are more overconfident than their pre-RLHF versions [2]. Leng et al. correct this during training. The method proposed here works at inference with any model and keeps the model's self-assessment out of the score, although bias can still reach the extraction step, which Section 6 measures.

Asking the model to explain its reasoning does not fix this. A written rationale is more text, not a measurement, and two rationales cannot be compared on a numeric scale. In regulated domains the problem is practical as well as statistical. A payer that auto-accepts a diagnosis code at "95% confidence" must be able to show an auditor why, and must know that the 95% means something.

Medical coding makes the stakes concrete. Soroush et al. [3] found that GPT-4, the best model tested, reproduced the exact ICD-10-CM code less than half the time. Accuracy will improve, but no

accuracy level removes the need to know which individual outputs to trust. Their study generated codes from code descriptions; this paper addresses a different task, scoring a candidate code against the full chart, and does not aim to improve coding accuracy itself.

Contributions. This paper makes three contributions:

1. A method that decomposes an LLM decision into evidence questions with cited, machine-verified answers, scored by deterministic code (Section 3).
2. A Bayesian interpretation in which rubric weights are log likelihood ratios, which makes the score a posterior probability once weights are fitted (Section 4).
3. A calibration and evaluation protocol that tests whether a stated probability matches observed accuracy (Section 6), with a full algorithm specification (Appendix A).

1.1 What the method adds to LLM validation

The method targets the two main failures of current LLM validation: *opacity*, meaning no account of why an output was produced, and *miscalibration*, meaning no defined meaning behind the confidence number. It is designed to add four things; Section 6 describes how each will be tested.

1. **Calibrated confidence instead of a verbalized guess.** Verbalized confidence clusters between 80% and 100% and is poorly calibrated against observed accuracy [1]. Here the score is a sum of log likelihood ratios with weights fitted by logistic regression, so it is a probability that can be tested against outcomes (Sections 4 and 6).
2. **An auditable trail instead of a black box.** Every score comes with the structured rubric answers and the source quotes behind them, and code checks each quote against the document. A reviewer can see exactly which evidence raised or lowered the score (Section 3).
3. **Silence and contradiction kept apart.** Binary checklist judges and yes/no checks merge "the source does not say" with "the source says the opposite." The three-way answer (yes, no, *not documented*) gives each its own weight. The distinction matters in clinical, legal and financial review, where a missing statement and a contrary statement lead to different decisions (Section 5, question Q3).
4. **Threshold routing on a real probability.** Because the score is calibrated, an organization can route on it, accepting high-scoring outputs automatically and sending the rest to human review. Where to set the thresholds is an operational choice made from calibration data, which balances throughput against compliance risk; this paper does not propose specific values.

2 Related work

Evidence-weighted confidence builds on the work summarized in Table 1; none of it alone yields an auditable, calibrated score for each LLM output. The last column states what this paper proposes to add. It is based on analysis and design and has not yet been validated experimentally. Each comparison is discussed further where the paper addresses it.

The proposed contribution is the combination: an LLM as the concept extractor, a verified citation for every concept, and a classical scorecard on top. The closest analogue is the concept bottleneck model, with the bottleneck made explicit, human-authored, and checked against the source document.

Table 1. Related work, its limits for this problem, and what this paper proposes to add.

Line of work	What it does	What it lacks for this problem	What this paper proposes to add
Verbalized and logit confidence [1, 2]; post-hoc recalibration [4]	Asks the model for a number, or reads token probabilities; Platt or isotonic mapping can recalibrate it	Overconfident, more so after RLHF; no evidence trail; cannot be decomposed or audited	A score computed from verified evidence instead of self-assessment; recalibrated verbalized confidence serves as a baseline
Checklist LLM judges, e.g. CheckEval [5]	Breaks evaluation into yes/no questions; improves agreement across judge models	Built for grading text quality; aggregation is a plain count, not a probability	Weights fitted to outcomes as log likelihood ratios; a third answer for *not documented*; quote verification
Concept bottleneck models [6]	Predicts human-readable concepts first, then the label from the concepts	Needs concept labels to train the concept predictor	A pre-trained LLM extracts concepts with cited quotes, so only outcome labels are needed (expected, not shown)
Weight-of-evidence scorecards and clinical scores [7]	Adds points per finding as log-odds; fitted by logistic regression	Assume structured inputs; no method for unstructured text	The same scoring theory applied to evidence an LLM extracts from text and code verifies
LLM medical coding benchmarks [3]	Measure how often a model produces the correct code	Overall accuracy does not say which individual outputs to trust	A per-output probability for a candidate code; no claim to improve coding accuracy
AI monitoring in prior authorization [8] (same author)	Label-based and label-free monitoring, a batch AI critic, a risk model for human scrutiny	Monitors the workflow over time; no calibrated probability per output	A per-output probability from criterion-level evidence, usable as an input to that monitoring

Status of these claims. The comparisons above come from analysis of the method's design, not from experiments. The evaluation in Section 6 is planned, and its results will be reported in a later version of this paper. If the method does not outperform an existing approach, for example recalibrated verbalized confidence, that result will be reported as well.

3 Method

The method separates *reading* from *judging*: the LLM reads the source and answers questions, and deterministic code turns the answers into a score (Figure 1).

Rubric. For each candidate output (here, a diagnosis code), a rubric defines a set of evidence questions. Each question has one of two roles:

- **Gate questions** must be answered yes for the output to be valid at all, like a prior-authorization criterion. A failed gate sets the score to zero.
- **Weighted questions** shift confidence up or down once the gates pass.

Each question has three possible answers: *yes*, *no* (the source contradicts it), and *not documented* (the source is silent). The three carry separate weights, because contradiction and silence are different evidence. Breaking a judgment into yes/no questions follows checklist evaluators such as CheckEval [5], which showed that it makes LLM judges agree more and their scores easier to interpret. CheckEval grades text quality and counts its answers; this method scores factual support against a source document, adds the third answer, and fits a weight to each answer.

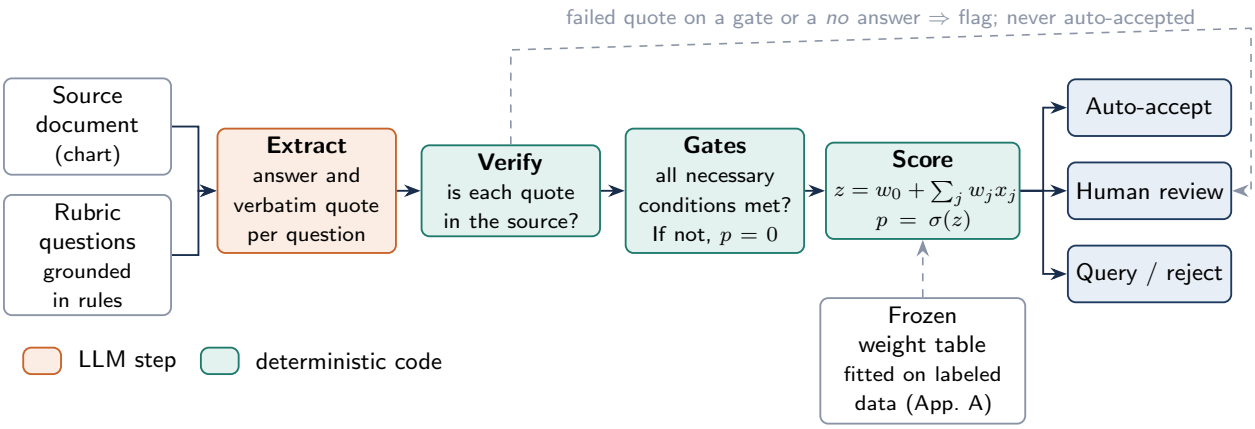


Figure 1. The evidence-weighted confidence pipeline. Only extraction uses an LLM; verification, gates, scoring and routing are deterministic and logged, so every score can be reproduced from its evidence.

Grounding. Questions are derived from the authoritative rules that govern the decision rather than written freely: the official coding guidelines for medical coding, the payer's medical policy for prior authorization, and payment policy for overpayment review. Each question records the rule it implements and that rule's version. The score then measures whether the documentation meets the rules the decision is held to, and a rubric change can be traced to a rule change. Section 5.1 shows the mapping for E11.22.

Pipeline.

1. **Extract.** The LLM receives the source document and the rubric questions, without the weights. For each question it returns an answer and, for yes or no, the exact quote that supports it.
2. **Verify.** Code checks every quote against the source with a plain string match: after case and spacing are normalized, the quote must appear word for word in the document. It is a string search, not a regular expression, because clinical text is full of characters (parentheses, periods, slashes, plus signs) that a regular expression would treat as commands. An answer whose quote is not found is downgraded to *not documented*. Downgrading is conservative only for evidence that would raise the score; for a contradiction it would remove a penalty. So when a gate answer or a *no* answer fails verification, the chart is also flagged for human review and cannot be auto-accepted, whatever its score.
3. **Score.** Code looks up the weight for each verified answer, adds the intercept, and converts the total from log-odds to a probability.
4. **Log.** The record stores answers, quotes, rubric version, weight-table version and score, so any score can be reproduced later.

An extraction record looks like this:

```
{"question": "ckd_stage_documented", "answer": "yes",
 "quote": "CKD stage 3a, stable", "verified": true}
```

The separation has three practical consequences. First, score variance across repeated runs comes only from extraction, which can be measured question by question. Second, a hallucinated piece of evidence fails verification instead of raising the score. Third, a new weight table can be applied to every historical record without new LLM calls, which makes calibration cheap. The

structure resembles a concept bottleneck model [6], which predicts human-readable concepts and then the label from them. The difference is the extractor: concept bottleneck models train a concept predictor on concept labels, while here a pre-trained LLM extracts the concepts with cited quotes. Only outcome labels are then needed to fit the weights, which should lower labeling cost; the pilot has not yet tested this.

4 Bayesian foundation

Adding points per finding is exact Bayesian updating when the points are log likelihood ratios. Let C be the event that the output is correct and $e_1, \ldots, e_n$ the verified answers. Bayes' rule in odds form is

$$\frac{P(C \mid e_1, \ldots, e_n)}{P(\neg C \mid e_1, \ldots, e_n)} = \frac{P(C)}{P(\neg C)} \prod_{i=1}^{n} \frac{P(e_i \mid C)}{P(e_i \mid \neg C)}. \tag{1}$$

The product holds when the answers are conditionally independent given C. Taking logs turns it into a sum:

$$\operatorname{logit} P(C \mid e) = w_0 + \sum_{i=1}^{n} w_i(e_i), \qquad w_0 = \log \frac{P(C)}{P(\neg C)}, \quad w_i(e_i) = \log \frac{P(e_i \mid C)}{P(e_i \mid \neg C)}. \tag{2}$$

The intercept w_0 is the prior: how often the output is correct before reading the evidence. Each weight w_i is the weight of evidence of one answer [7]. This is the theory behind weight-of-evidence scorecards and clinical point scores, which fit their points by logistic regression and commonly give missing values their own bin. Those tools assume structured inputs; the contribution here is applying them to evidence an LLM extracts from unstructured text and code verifies. The three-way answer adapts the missing-value bin rather than introducing a new idea. Four consequences follow.

1. **Scores compose without overflow.** Adding "+30%" per finding can exceed 100%. Log-odds are unbounded and map back to a probability between 0 and 1.
2. **Absence of evidence counts.** If a finding is common among correct outputs, its absence has a likelihood ratio below 1 and a negative weight. A rubric that only adds points for findings ignores this. With silence as the zero point (below), the same fact shows up as a larger weight for yes.
3. **Correlated evidence is double-counted.** Two answers that restate one fact (a CKD diagnosis in the problem list and in the assessment) violate independence and inflate the score. This is the known weakness of naive Bayes.
4. **Logistic regression fixes point 3.** Logistic regression has the same additive log-odds form but fits all weights jointly, so correlated answers share credit. Its coefficients are the calibrated weights.

The method therefore has two stages. Expert-assigned weights give a usable, auditable score on day one. Fitted weights turn that score into a probability that can be tested.

In practice the weights are written with silence as the zero point: every *not documented* answer counts 0, and its effect is absorbed into the starting score w_0. Predictions are unchanged, but w_0 then means the log-odds for a chart that passes the gates and documents nothing further, rather than the population base rate. This is the only form used in the rest of the paper, because it is the form logistic regression can fit uniquely (Appendix A).

5 Worked example: E11.22

E11.22 (type 2 diabetes mellitus with diabetic chronic kidney disease) shows gates, three-way answers and a contradiction in one rubric (Table 2). The weights are illustrative, not fitted. Silence (*not documented*) counts 0 on every question. The starting score is $w_0 = -4.30$: the log-odds for a chart that passes the gates but documents nothing beyond them. Each answer then adds to or subtracts from that starting score.

Table 2. Illustrative E11.22 rubric. Weights are added to the starting score $w_0 = -4.30$; silence is the zero point.

Question	Role	Yes	No (contradicted)	Not documented
G1. Type 2 diabetes documented in the encounter note	Gate	pass	fail	fail
G2. Chronic kidney disease documented	Gate	pass	fail	fail
Q1. CKD stage documented	Weighted	+2.19	n/a	0
Q2. eGFR below 60 on two results 90+ days apart	Weighted	+1.40	−1.20	0
Q3. Provider attributes CKD to diabetes	Weighted	+0.90	−2.00	0
Q4. Kidney disease monitored, evaluated, assessed or treated this encounter (MEAT)	Weighted	+2.20	n/a	0

Q3 shows why three answers matter. ICD-10-CM's "with" convention presumes a link between diabetes and CKD unless the provider documents another cause, so silence costs nothing (0). An explicit statement adds evidence (+0.90). A documented other cause, such as hypertensive kidney disease, is strong evidence against the code (−2.00).

5.1 Grounding the rubric in the coding guidelines

Every E11.22 question implements a specific rule from the ICD-10-CM Official Guidelines for Coding and Reporting (FY 2027, effective October 1, 2026) [9] or from the code's instructional note in the Tabular List [10]. Under HIPAA, adherence to these guidelines is required when assigning diagnosis codes [9]. A threshold on the score therefore measures how well the documentation meets rules coders are already held to, rather than how closely the note's wording resembles a code description (Table 3).

Table 3. Source of each E11.22 rubric question.

Question	Source	What the source says	How the question applies it
G1. Type 2 diabetes documented	Categories E10 and E11; Section I.C.4.a [9]	Diabetes codes are combination codes classified by type: E11 is type 2, E10 is type 1. When the type is not documented, the default is E11	A type 1 chart belongs under E10.22, so the gate fails. Diabetes documented without a type counts as yes

Table 3 continued

Question	Source	What the source says	How the question applies it
G2. CKD documented as an established diagnosis	Sections IV.H, IV.J and I.B.14 [9]	Outpatient: "Do not code diagnoses documented as 'probable', 'suspected,' 'questionable,' 'rule out' " or similar terms. Code coexisting conditions that affect care, but not "conditions that were previously treated and no longer exist." Code assignment is based on the provider's documentation	Possible or rule-out CKD fails. Resolved acute kidney injury fails. Low eGFR values without a provider's CKD diagnosis fail
Q1. CKD stage documented	E11.22 instructional note [10]	"Use additional code to identify stage of chronic kidney disease (N18.1–N18.6)"	Rewards a documented stage. E11.22 remains assignable without one, so Q1 is weighted rather than a gate
Q2. eGFR below 60 on two results 90+ days apart	Section I.A.19 [9]	"Code assignment is not based on clinical criteria used by the provider to establish the diagnosis"	Lab support is not a coding requirement, so Q2 is weighted rather than a gate. It reflects clinical validation, a separate review that payers and auditors apply
Q3. Provider attributes CKD to diabetes	Section I.A.15, "With" [9]	The classification "presumes a causal relationship" between conditions linked by "with", even without explicit provider linkage, "unless the documentation clearly states the conditions are unrelated"	Silence costs nothing (weight 0). An explicit link adds evidence. A documented other cause rebuts the presumption and carries a strong negative weight
Q4. CKD addressed this encounter	Sections IV.I and IV.J [9]; MEAT audit convention	Chronic diseases "treated on an ongoing basis may be coded and reported as many times as the patient receives treatment and care"; code conditions "that require or affect patient care, treatment or management"	Requires evidence that CKD was monitored, evaluated, assessed or treated at this visit. MEAT is an industry risk-adjustment audit convention, not text from the Official Guidelines

Three rules shape the E11.22 rubric most:

1. **Linkage is presumed unless contradicted.** Section I.A.15 presumes a causal relationship between diabetes and chronic kidney disease when they occur "with" each other. The presumption holds unless the documentation clearly states the conditions are unrelated. Q3 tests whether the link is present, absent or contradicted, and only a contradiction counts against the code.
2. **The code calls for a stage.** The instructional note for E11.22 requires an additional code for the CKD stage from N18.1–N18.6. Q1 checks whether the note documents the stage needed for that companion code.
3. **The diagnosis must be certain and current.** Outpatient guidelines exclude uncertain diagnoses and conditions that no longer exist, and code assignment rests on the provider's documentation. G2 therefore rejects rule-out impressions, resolved conditions and diagnoses inferred from labs alone.

The mapping is the author's reading of the guidelines. It has not yet been reviewed by a certified coder, and payer policy or official coding advice may add requirements.

5.2 Three charts

Table 4 scores three charts that all pass the gates, and Figure 2 places them on the logistic curve.

Table 4. Three charts scored with the illustrative weights of Table 2.

Chart	Verified answers	Log-odds	Probability
A. Complete documentation	Q1 yes, Q2 yes, Q3 yes, Q4 yes	$-4.30 + 2.19 + 1.40 + 0.90 + 2.20 = 2.39$	0.92
B. Problem-list mention only	Q1 yes, Q2 n.d., Q3 n.d., Q4 n.d.	$-4.30 + 2.19 + 0 + 0 + 0 = -2.11$	0.11
C. CKD attributed to hypertension	Q1 yes, Q2 yes, Q3 no, Q4 yes	$-4.30 + 2.19 + 1.40 - 2.00 + 2.20 = -0.51$	0.38

n.d. = not documented.

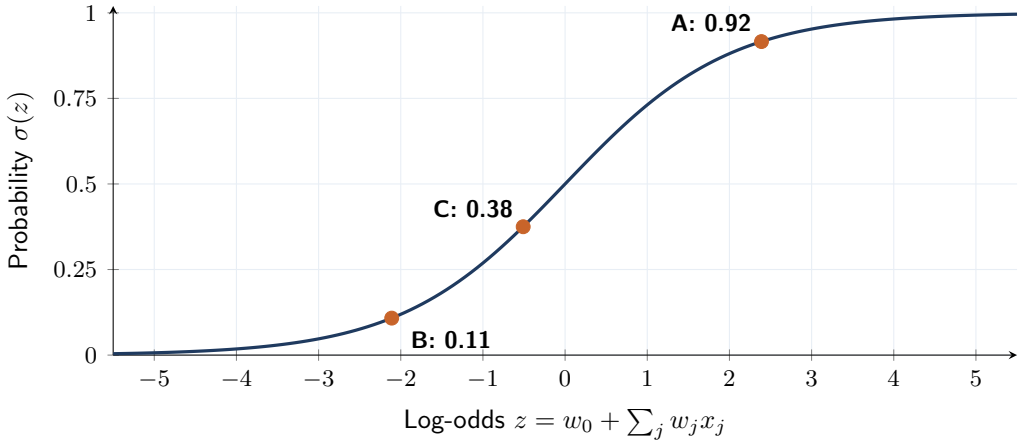


Figure 2. The three charts of Table 4 on the logistic curve. Each chart's evidence adds up to a log-odds score z, and the sigmoid turns it into a probability. The same change in evidence moves the probability most in the middle of the curve and least at the extremes.

An LLM asked for a confidence number might rate all three charts 85–95%, because each mentions both diabetes and CKD. The rubric separates them, and each score comes with the quotes that produced it. Chart B is the typical unsupported diagnosis in a risk-adjustment audit: carried in the problem list but not addressed in the visit. Chart C would likely be coded differently by a human coder.

With calibrated weights, the score supports threshold routing (Section 1.1). Thresholds are an operational choice made from calibration data to meet a target precision and reviewer workload; this paper does not propose specific values.

6 Calibration and evaluation protocol

The claim that a score is a probability is falsifiable, and this protocol tests it.

Data. A labeled set of (document, candidate code, correct or not) triples, labeled by certified coders, with inter-rater agreement reported. Public options include synthetic notes or de-identified notes such as MIMIC; a rough sample-size rule is 10–20 positive cases per fitted weight.

Fitting.

1. Run extraction once on all documents and store the verified answers.
2. Encode each answer as indicator features: yes and no per question, with *not documented* as the zero point.
3. Fit L2-regularized logistic regression on a training split. For rare codes, fit at the family level and let each code deviate slightly (partial pooling; see Section 7).
4. Score the held-out split with the fitted weight table.

Appendix A specifies the encoding, objective, regularization search and prior adjustment precisely enough to reimplement.

Metrics. Table 5 lists what is measured.

Table 5. Evaluation metrics.

Property	**Metric**
Calibration	Expected calibration error (ECE), Brier score, reliability diagram
Discrimination	AUROC
Operational value	Share of outputs auto-accepted at a target precision (coverage at precision)
Decision quality	Precision (headline), recall and $F_{0.5}$ at the auto-accept threshold chosen on the development split and applied unchanged to the test split, and at a fixed 0.5 threshold
Extraction quality	Criterion level, per question: precision and recall for each answer type, macro-F_1, and a confusion matrix against reference answers
Reproducibility	Score standard deviation over repeated extraction runs of the same document
Faithfulness	Share of quotes that fail verification, overall and per question; share of yes answers a coder rejects

Plain accuracy is not used as a headline metric. For the final decision, calibration and AUROC evaluate the probability itself, and threshold metrics are reported only at fixed operating points. $F_{0.5}$ is chosen because it weights precision over recall: for a payer, accepting an unsupported code risks recovery in an audit, while missing a supported one costs less. A use case with the opposite priority would report F_2. For extraction, answer types are imbalanced (in the pilot data, a documented contradiction is far rarer than silence), so a model that always answered *not documented* could score well on agreement while missing every contradiction. Per-class recall and macro-F_1 expose that failure.

Baselines.

1. Verbalized confidence: the model reports a number directly.
2. Verbalized confidence after post-hoc Platt [4] or isotonic recalibration, the strongest existing comparison. Platt scaling maps one score to a probability after the fact; the method instead fits a weight to each piece of evidence, which keeps the score decomposable.
3. Self-consistency: agreement rate across several sampled answers.
4. The rubric with expert weights, unfitted.
5. The rubric with the LLM computing the score itself, to isolate the effect of deterministic scoring.

Ablations. Remove span verification; collapse three-way answers to two; drop gates; fit naive Bayes instead of logistic regression to measure the cost of the independence assumption.

The central hypothesis is that the fitted rubric matches or beats recalibrated verbalized confidence on ECE and AUROC while adding a verified evidence trail. If it matches without beating, the evidence trail alone justifies it in audited settings.

7 Generalizing across use cases

The rubric changes with every use case; the pipeline, the scoring theory and the protocol do not (Table 6).

Table 6. The same structure applied to other decisions.

Use case	Output being scored	Example gate	Example weighted question
Diagnosis coding	An ICD-10-CM code	Condition documented in a face-to-face encounter	Condition treated or monitored this visit
Prior authorization	An approve recommendation	Diagnosis on the policy's covered list	Six weeks of failed conservative therapy documented
Post-pay overpayment	An overpayment finding	Claim falls under the cited policy	Billed units exceed the documented service
Clinical-trial matching	A patient-trial match	Age within the inclusion range	Lab values within the inclusion window

Prior authorization is the closest analogue. Teams already turn policy lines into questions and have an LLM answer them with citations. Prior authorization combines the answers with AND/OR logic into approve, deny or pend. This method keeps that logic for gates and adds weighted questions for graded confidence, so an existing policy-to-question engine can be reused with only the combination layer changed. The same criterion-level structure supports monitoring such a system in production [8].

Scaling the rubrics. Writing a separate rubric for each of roughly 70,000 ICD-10-CM codes is not feasible. Rubrics are layered instead:

1. **Generic layer:** questions that apply to every output of the use case (for coding, the MEAT criteria and encounter-type rules).
2. **Family layer:** shared logic for a category (for diabetes with complications: condition, complication, link).
3. **Code layer:** overrides for high-value or high-error codes only.

The layering is also a statistical device. A rare code with 15 labeled examples cannot support its own weights, but it can borrow its family's weights and adjust slightly: partial pooling, as in hierarchical regression. Effort goes first to the small set of codes that drive most financial value and audit risk. Candidate questions can be drafted by an LLM from official coding guidelines and then reviewed by certified coders, with every rubric versioned.

8 Extensions for improving accuracy

The method is a way to measure confidence; the extensions below aim to make what it measures more accurate or cheaper to obtain. None is required by the method, and none is part of the planned

pilot. They apply to any use case, not only medical coding.

- **Semantic verification of evidence.** Exact-quote matching shows that the evidence exists, not that it supports the answer. A second model, ideally from a different model family, could judge whether each quote, shown with its surrounding text and section, supports, fails to support or contradicts the answer. This is the attribution-checking problem studied for LLM-generated text [11–13], and it corresponds to the conservative AI critic proposed for prior-authorization monitoring in [8]. Answers the verifier rejects would follow the same rule as failed quotes: downgrade evidence that raises the score, and send contradictions and gate answers to human review.
- **Decision-relevant verification.** Because scoring is deterministic and nearly free, costly checks can be limited to answers whose removal would move an output across a routing threshold. A random sample of skipped checks would still be run, to measure how often skipping is wrong.
- **Cost controls.** Asking all questions in one call per document, caching the fixed instructions and rubric, requiring compact structured output, and batch processing all reduce cost without changing the scoring. Shortening the model's rationale should be tested, since it may reduce accuracy on hard cases.
- **Evidence retrieval across documents.** Many reviews span several documents, such as earlier visits or separate lab reports. An orchestrating agent could fetch additional records when evidence is missing, with every action logged. The scoring and routing would stay fixed, so the score remains reproducible from the logged evidence.
- **Production monitoring.** Once deployed, labels are scarce. The method's intermediate outputs give label-free signals in the sense of [8]: the rate of each answer per question, the quote-failure rate, the gate-failure rate and the share of charts flagged for review. A shift in these signals shows where the system changed, though not whether it is wrong. Periodic human review of a targeted sample then gives labels to recheck calibration and criterion-level extraction quality.

9 Limitations

- **Documented is not the same as true.** The score measures how well the source supports an output. Cloned or copied-forward notes can satisfy every question.
- **Extraction errors remain.** Verification catches invented quotes, not real quotes misread. A quote can exist and still not support the answer.
- **Rubric bias.** Questions and expert weights encode their authors' views. Fitting corrects the weights, not missing questions.
- **Guideline interpretation.** The rubric encodes one reading of the coding guidelines. Coders, payers and auditors can read edge cases differently, such as a missing CKD stage or a copied-forward note, and the mapping needs review by certified coders.
- **Label cost.** Calibration needs coder-labeled data per code family, and labels themselves carry disagreement.
- **Drift.** Documentation habits, coding guidelines and risk-adjustment models change, so weights must be refitted and rubrics re-versioned.

- **Gaming.** If providers learn the rubric, documentation may be written to satisfy it rather than to describe care.

10 Conclusion

A confidence number has meaning only when it is computed from checked evidence and tested against outcomes. Evidence-weighted confidence is designed to do both: the LLM supplies cited answers to domain questions, code verifies and scores them, and logistic regression turns the score into a calibrated probability. The additive scoring is exact Bayesian updating in log-odds form. Together these are intended to give LLM validation four things it lacks: calibrated probabilities, a verified audit trail, separate treatment of silence and contradiction, and threshold routing that trades throughput against compliance risk. The rubric is specific to each use case, but the theory and the evaluation protocol carry across. The next step is the experiment in Section 6 on a labeled coding dataset; its results will follow in a later version.

References

[1] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2306.13063.

[2] Jixuan Leng, Chengsong Huang, Banghua Zhu, and Jiaxin Huang. Taming overconfidence in LLMs: Reward calibration in RLHF. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.09724.

[3] Ali Soroush et al. Large language models are poor medical coders: Benchmarking of medical code querying. *NEJM AI*, 1(5), 2024. doi: 10.1056/AIdbp2300040.

[4] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, editors, *Advances in Large Margin Classifiers*. MIT Press, Cambridge, MA, 2000.

[5] Yukyung Lee, Joonghoon Kim, Jaehee Kim, Hyowon Cho, Jaewook Kang, Pilsung Kang, and Najoung Kim. CheckEval: A reliable LLM-as-a-judge framework for evaluating text generation using checklists. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. URL https://arxiv.org/abs/2403.18771.

[6] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 5338–5348, 2020. URL https://proceedings.mlr.press/v119/koh20a.html.

[7] I. J. Good. *Probability and the Weighing of Evidence*. Griffin, London, 1950.

[8] Issam Zeinoun. Beyond accuracy: A risk-driven architecture for monitoring AI in prior authorization. Zenodo, 2026. URL https://doi.org/10.5281/zenodo.23628695. Also posted on LinkedIn, September 2026.

[9] Centers for Medicare & Medicaid Services and National Center for Health Statistics. ICD-10-CM official guidelines for coding and reporting, FY 2027, 2026.

URL https://ftp.cdc.gov/pub/Health_Statistics/NCHS/Publications/ICD10CM/2027/ICD-10-CM-October-1-2026-FY27-Guidelines.pdf. Effective October 1, 2026.

[10] Centers for Medicare & Medicaid Services. 2027 ICD-10-CM code tables, tabular and index, 2026. URL https://www.cms.gov/files/zip/2027-code-tables-tabular-index.zip. Tabular List entry for E11.22, Type 2 diabetes mellitus with diabetic chronic kidney disease, and its instructional notes.

[11] Hannah Rashkin, Vitaly Nikolaev, et al. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4), 2023. URL https://aclanthology.org/2023.cl-4.2.

[12] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6465–6488, 2023. URL https://aclanthology.org/2023.emnlp-main.398.

[13] Xiang Yue, Boshi Wang, Ziru Chen, Kai Zhang, Yu Su, and Huan Sun. Automatic evaluation of attribution by large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL https://arxiv.org/abs/2305.06311.

A Algorithm specification

The pipeline has two paths. Inference scores one chart with a frozen weight table; training fits that table from labeled charts. Extraction is the only step that uses an LLM; everything after it is deterministic code.

A.1 Notation

- T: the source document (chart text).
- Gate questions $g_1, \ldots, g_k$ and weighted questions $q_1, \ldots, q_m$, each answered from the set $\{yes, no, not_documented\}$.
- Labeled data $\mathcal{D} = \{(T_i, y_i)\}_{i=1}^{N}$, with $y_i = 1$ if the candidate code is supported and 0 otherwise.

A.2 Inference

Algorithm 1 Scoring one document with a frozen weight table

Require: document T; gates G; weighted questions Q; weights (w_0, w)

$\{(a_j, s_j)\} \leftarrow$ LLM-Extract$(T, G \cup Q)$ ▷ answers and verbatim quotes; weights never shown
review $\leftarrow$ **false**
for all questions j with $a_j \in \{yes, no\}$ **do**
  if Normalize(s_j) is not a substring of Normalize(T) **then**
    if $j \in G$ **or** $a_j = no$ **then**
      review $\leftarrow$ **true**
    end if
    $a_j \leftarrow$ *not_documented* ▷ downgrade and log
  end if
end for
if $\exists\, g \in G : a_g \neq yes$ **then**
  return $p = 0$, *review*
end if
$x \leftarrow$ indicators $(\mathbb{1}[a_q = yes], \mathbb{1}[a_q = no])_{q \in Q}$ ▷ *not_documented* is the reference
$z \leftarrow w_0 + w^\top x; \quad p \leftarrow 1/(1 + e^{-z})$
log answers, quotes, verification results, rubric and weight versions, z, p
return p, *review* ▷ flagged charts are never auto-accepted

Verification is a plain substring test after normalization (case folding, unified quote and dash characters, and collapsed whitespace); no regular expressions or fuzzy matching are used. Reference coding matters for interpretation. With one indicator per answer plus an intercept, a question's three indicators always sum to 1, so the weights are not uniquely determined: a constant can move between them and the intercept without changing any prediction. With *not_documented* as the baseline, each weight is the change in log-odds relative to the source being silent. The illustrative weights in Section 5 use the same convention. The probability is

$$z = w_0 + \sum_{j=1}^{d} w_j x_j, \qquad P(Y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad d \leq 2m. \tag{3}$$

A.3 Training

Only charts that pass every gate enter training, because gate failures are scored 0 by rule and carry no information about the weights. The weights minimize L2-regularized log-loss, with the intercept

w_0 unpenalized:

$$\min_{w_0,\,w} \; \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{N}\Big[-y_i\log\sigma(z_i) - (1-y_i)\log\big(1-\sigma(z_i)\big)\Big], \qquad z_i = w_0 + w^\top x_i. \tag{4}$$

- **Log-loss** is a proper scoring rule: its expected value is lowest only when the predicted probabilities are the true ones. That is why it serves as both the training objective and the model-selection criterion.
- **The L2 penalty** keeps weights finite when an indicator perfectly separates the classes, which is likely in small or synthetic datasets. Without it, the fitted weights would grow without bound.
- $\boldsymbol{C}$ is the inverse penalty strength. Smaller C gives smaller, more conservative weights; larger C fits the training data more closely.
- **The intercept is not penalized**, so the prior is not pulled toward 50%. The objective is convex, so any standard solver (for example L-BFGS) reaches the same unique solution.

Gradient. Write $L(w_0, w)$ for the objective in Equation (4). Because the derivative of the sigmoid cancels against the log-loss, the gradient is short:

$$\frac{\partial L}{\partial w_j} = w_j + C\sum_{i=1}^{N}\big(\sigma(z_i) - y_i\big)\,x_{ij}, \qquad \frac{\partial L}{\partial w_0} = C\sum_{i=1}^{N}\big(\sigma(z_i) - y_i\big). \tag{5}$$

A plain gradient-descent iteration moves every weight a small step η against its gradient:

$$w \leftarrow w - \eta\,\nabla_w L, \qquad w_0 \leftarrow w_0 - \eta\,\frac{\partial L}{\partial w_0}. \tag{6}$$

Each weight therefore moves in proportion to the prediction error $\sigma(z_i) - y_i$, summed over the charts where its answer appeared. If charts with Q3 = yes keep scoring lower than their labels, the Q3-yes weight rises. The penalty term w_j pulls every weight back toward 0; the intercept has no such term, so the prior is not shrunk. The reference implementation uses L-BFGS, a faster method driven by the same gradient. Because the objective is convex, it has a single minimum, so gradient descent, L-BFGS and any other standard solver reach the same weight table from any starting point.

The selection of C in Algorithm 2 is k-fold cross-validated grid search, not nested cross-validation. Nested cross-validation adds an outer loop that estimates how well the whole selection procedure generalizes; here the untouched test split plays that role. When the loss keeps improving as C grows, the weights reflect the penalty rather than measured evidence strength.

A.4 Adjusting the prior for deployment

The fitted intercept reflects the share of positive charts in the training data. When the deployment population has a different share, shift the intercept:

$$w_0' = w_0 + \operatorname{logit}(\pi_{\text{deploy}}) - \operatorname{logit}(\pi_{\text{train}}), \qquad \operatorname{logit}(p) = \log\frac{p}{1-p}. \tag{7}$$

Here π is the share of positive charts among those that pass the gates. The correction assumes the evidence looks the same within each class in both settings (label shift only), so calibration must still be checked on a deployment sample. This matters for the pilot: its synthetic charts are 44% positive, far above a real diabetic population.

Algorithm 2 Fitting the weight table on the development split

Require: development charts with verified answers and labels; target grid $\mathcal{C} = \{10^{-3}, 10^{-2.5}, \ldots, 10^{3}\}$ (13 values)

keep only charts whose gate answers are all *yes*; encode x_i as in Algorithm 1
for all $C \in \mathcal{C}$ **do**
  for $r = 1, \ldots, 10$ **do** ▷ repeated, stratified 5-fold CV; a patient's charts stay in one fold
    for each fold $f = 1, \ldots, 5$ **do**
      fit Equation (4) on the other four folds; record log-loss on fold f
    end for
  end for
  $L(C) \leftarrow$ mean log-loss over the 50 fits
end for
$C^{\star} \leftarrow \arg\min_C L(C)$ ▷ or the one-standard-error rule, fixed in advance
if $C^{\star}$ lies at the edge of $\mathcal{C}$ **then**
  extend the grid; if the loss keeps improving, report that the indicators separate the classes
end if
refit on all development charts with $C^{\star}$; freeze and version the weight table
score the held-out test split once (Section 6)

A.5 Reference implementation

A minimal Python version of the encoding and of Algorithm 2, using scikit-learn (tested with version 1.9.1, whose L-BFGS solver leaves the intercept unpenalized):

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, RepeatedStratifiedKFold

WEIGHTED_QS = ["Q1", "Q2", "Q3", "Q4"]
GATES = ["G1", "G2"]

def encode(answers):
    """Verified answers -> indicators; not_documented is the reference category."""
    return [int(answers[q] == v) for q in WEIGHTED_QS for v in ("yes", "no")]

def fit_weights(dev_answers, dev_labels):
    keep = [i for i, a in enumerate(dev_answers) if all(a[g] == "yes" for g in GATES)]
    X = np.array([encode(dev_answers[i]) for i in keep])
    y = np.array([dev_labels[i] for i in keep])
    search = GridSearchCV(
        LogisticRegression(solver="lbfgs", max_iter=10_000),  # L2 by default
        param_grid={"C": np.logspace(-3, 3, 13)},
        scoring="neg_log_loss",
        cv=RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0),
        refit=True,
    )
    search.fit(X, y)
    model = search.best_estimator_
    names = [f"{q}_{v}" for q in WEIGHTED_QS for v in ("yes", "no")]
    table = {"intercept": float(model.intercept_[0]),
             **dict(zip(names, map(float, model.coef_[0])))}
    return table, search.best_params_["C"]
```